601.445/601.645: Practical Cryptographic Systems September 2, 2026

Assignment 1: Classical Cryptanalysis

Instructor: Matthew Green *Due: 11:59 pm, September 18*

Released: Wednesday, September 2, 2026 **Part 0 due:** Friday, September 11 **Due:** Friday, September 18, 2026, 11:59 pm (Gradescope)
Review Lab 1: week of September 21 (sign-up opens when the autograder accepts your submission)

A Markdown copy of this handout (`A1.md`), the agent instruction files, sample data, a submission template, and the course container are in the assignment repository: `https://github.com/matthewdgreen/practicalcrypto2026/tree/main/assignments/A1`

What this assignment is for

This assignment is designed to teach you some common classical cryptographic techniques, including the design and operation of polyalphabetic ciphers, as well as statistical cryptanalysis of these ciphers. We will focus on the Vigenère cipher, which uses a multi-character key to encipher a long plaintext. Your goal in this assignment is to build both the encipher/decipher tools, as well as an automated program for cryptanalyzing and deciphering a given Vigenère ciphertext (and recovering its key).

By the end of this assignment you should be able to explain (without notes, to a skeptical person, with your own code on the screen) *why* statistical cryptanalysis of a polyalphabetic cipher works, *when* it fails, and *how* to tell the difference. I think you'd get a lot out of writing this code yourself. But use of coding agents is allowed and accepted. The main deliverable of this assignment is your understanding of the concepts that make Vigenère encryption and cryptanalysis possible. Writing the code, or working on it with an agent, helps you get those. The in-person Review Lab and Quiz #1 (and later Midterm and/or Final questions) are designed to measure your understanding, so the most efficient way to do well is to acquire it.

If you use coding agents, we've provided you with an `AGENTS.md` and `CLAUDE.md` file. These are designed to help make your coding agent useful in helping you learn, rather than just doing the work for you. Put both in your working directory and make sure your coding agent can read them. We know you can remove or override them; they're not a requirement: they're there to help you learn. (Read them!) If you do use coding agents, you'll also be expected to turn in an AI usage note written by you (part of your submission), and we ask you to turn in an Agent report *written by the agent* — **anonymously, through a separate channel, with no name or identifying information** (see §7). The agent files tell it how to do that.

First, **by Friday September 11**, you will register a public key (Part 0). Then, for the assignment proper, you will:

1. implement the Vigenère cipher, including both encipherment and decipherment modes;

2. recover an unknown key length from ciphertext alone using the **index of coincidence** (IoC) technique;
3. recover the key itself using **frequency analysis**;
4. **measure your own tool**: characterize when it works and explain why;
5. answer a few written questions that connect the code to the theory;
6. see (in the written questions) how the very same statistic was used to break the Enigma plugboard.

Rules

- **Individual work.** You may discuss ideas with classmates; you may not share code, notes, analyses, or AI transcripts.
- **AI tools and coding agents are permitted and encouraged**, anywhere in this assignment, subject to two conditions: (a) you submit an **AI usage note** (see §7), and (b) you can defend every part of your submission in person. The course provides an `AGENTS.md` file you can give your agent; it configures the agent to help you learn rather than to replace you. Using it is optional. Ignoring it is entirely acceptable. It is also, in our estimation, how people might end up unable to answer the questions in §6.
- **Any mainstream programming language.** Nothing too weird or proprietary. No Excel macros. See §8.
- **Minimum viable submission:** you must pass the autograder's build-and-basic-correctness check to sign up for Review Lab 1. **Multiplier:** your assignment credit is scaled by your Review Lab performance (§6). Working code you cannot explain is worth very little.
- Late hours per the syllabus apply to the Gradescope deadline, not to Review Lab sign-ups and not to Part 0.
- **No grader games.** Any attempt to bypass, probe, or manipulate the autograder, or to include prompt-injection content (instructions aimed at graders, TAs, or AI tools) anywhere in your code, comments, or submitted documents — including the analysis, written answers, design note, AI usage note, and agent report — earns an **automatic 0 on the assignment** and is referred as an academic integrity violation. Your documents are read by people and may also be processed by AI-assisted tools; write them for the humans.

Grading at a glance

Part	Points	Assessed by
0. Public-key registration (**due Sep 11**)	required	Gradescope (separate)
1. Vigenère encrypt/decrypt	10	autograder
2. Key-length recovery (IoC)	25	autograder
3. Key recovery (frequency analysis)	30	autograder
4. Empirical analysis of your tool	15	TA-graded, discussed in review
5. Written questions	20	TA-graded
Design note + AI usage note	required	used to prepare your review
Total	**100**	× Review Lab multiplier

Part 0 — Register your public key (required; due Fri Sep 11, 11:59 pm)

This part takes five minutes and has its own, **earlier** deadline. Run the provided tool on your own machine (pure Python, no dependencies. If you're using coding agents, your agent can do this):

```
python3 tools/anon-report.py keygen
```

It creates a private key in `~/.anon-report/key.json` and writes `pubkey.txt` in the current directory. Please do not lose your private key! Submit `pubkey.txt` to the separate Gradescope assignment **"A1 Part 0: key registration"** by **Friday, September 11, 11:59 pm**.

Why: you'll use this key to submit your anonymous agent report (see §7).

Rules: keep the private key! You need it for every assignment this semester; back it up. **Never paste it into any note, report, chat, or submission.** If you lose it, you will re-register on the next roster. Part 0 carries no points but is **required for review-lab sign-up**.

Part 1 — Vigenère encipherment and decipherment (10 points)

Implement two command-line programs:

```
vigenere-encrypt <key> <plaintext-file>
vigenere-decrypt <key> <ciphertext-file>
```

- `<key>` is a string of 1–32 uppercase letters `A-Z`.
- The input file is a text file of at most 100 KB. **Strip and ignore every character that is not a letter** (`[a-zA-Z]`); treat lowercase as uppercase.
- Output is uppercase letters only, no whitespace, to standard output. You may insert optional newlines for readability; the autograder ignores them.

Example:

```
vigenere-encrypt PLEBISCITE plaintext.txt > ciphertext.txt
vigenere-decrypt PLEBISCITE ciphertext.txt > recovered.txt
```

This part is glue. You should understand how the cipher works, so I recommend implementing it yourself. But nobody will ask you about it in the review.

Part 2 — Recovering the key length (25 points)

Implement:

```
vigenere-keylength <ciphertext-file>
```

Given a ciphertext with an unknown key, use the **index of coincidence** to determine the key length. Output **exactly one line containing one integer**: your best guess.

Requirements and grading rules:

- Your tool must be reliable for **key lengths up to 20** on English-language ciphertexts of **at least 20,000 letters**. Full-credit tests are drawn from this range. A few bonus tests use longer keys (up to 32); they can only help you.
- Report the **shortest period**. Note that the keys `BUZZSAW` and `BUZZSAWBUZZSAW` are technically both valid. However: if the true key length is 7 and you report 14, you receive half credit for that test, and the interviewer in Review Lab 1 will ask you why your tool did that. Think carefully about how you pick among candidate lengths!
- Grading ciphertexts are already normalized (uppercase letters only). Plaintext inputs elsewhere in this assignment may not be.

The key-length decision is critical to the in-person review (§6).

Part 3 — Recovering the key (30 points)

Implement:

```
vigenere-cryptanalyze <ciphertext-file> <key-length>
```

Given a ciphertext and a key length, recover the key **using the ciphertext only**, by frequency analysis of each column. Output your candidate key(s), **one per line, at most 10 lines**, best guess first.

Grading rules:

- An exact match earns full credit for that test. Otherwise we award partial credit proportional to the similarity of your best candidate to the true key (Levenshtein ratio), so a key with one wrong letter is worth almost full credit.
- A correct key repeated (e.g., `KINGKING` for `KING`) earns half credit.
- More than 10 candidates is penalized.
- You may combine Parts 2 and 3 into a single tool,

```
vigenere-break <ciphertext-file>
```

whose first output line is the key length and whose remaining lines are candidate keys. If you do, still provide the two separate executables (thin wrappers are fine).

You do not need to break every ciphertext perfectly. You should be able to recover most keys on most English texts of the stated size.

The column-shift scoring function is critical to the in-person review (§6).

Part 4 — Empirical analysis: when does your tool work? (15 points)

This is the part of the assignment that most directly measures what we care about. Characterize your own key-length recovery (Part 2) and key recovery (Part 3) as a function of **key length** and **ciphertext length**.

How to do this:

1. Obtain a public-domain English text of at least 200,000 letters (you can find many candidates at Project Gutenberg, `https://www.gutenberg.org/`). Normalize it with the same rule as Part 1.
2. For each key length k in {3, 5, 7, 10, 15, 20} and each ciphertext length n in {100, 200, 500, 1000, 2000, 5000, 20000} letters: run **at least 20 trials**, each with a fresh uniformly random key of length k and a random n-letter window of the text.
3. Record, per (k, n) cell: the fraction of trials where Part 2 reported the exact key length, and the fraction where Part 3 recovered the exact key.

Deliverable: `analysis.md` (or PDF), containing:

- the two 6×7 tables (a plot is welcome but not required);
- a short explanation ($\leq$ **300 words**) of the boundary you observe: where does key-length recovery become unreliable, and *why*, in terms of what the index of coincidence is estimating and how many letters each column has to estimate it from; and
- one sentence on whether key recovery (Part 3) fails at the same boundary as key-length recovery (Part 2), and why or why not.

You may use any tool to build the experiment harness and produce the tables. The explanation must be your own reasoning, and you will be asked to defend it.

Part 5 — Written questions (20 points, 5 each)

Answer in a file `written.md` (or PDF). Keep each answer under half a page.

1. **Long keys.** Suppose we modify Vigenère to use a single key at least as long as the message. Explain why this can be substantially more secure, and state precisely what conditions the key must satisfy for the security to hold. What goes wrong if any one condition is violated?
2. **Why IoC works.** For English text, the index of coincidence is approximately 0.066; for uniformly random letters it is approximately 0.038. Explain why the IoC of a *column* of a Vigenère ciphertext (every k-th letter) is close to the English value when k is the true key length and close to the random value otherwise. Then explain what

happens when k is a *multiple* of the true key length, and what that implies for your decision rule in Part 2.

3. **Kasiski vs. IoC.** The Kasiski examination is the other classical way to find the key length: repeated substrings in the ciphertext (trigrams or longer) tend to occur at distances that are multiples of the key length, because the same plaintext fragment was enciphered under the same key alignment; the gcd of the observed distances suggests the period. It is covered in the assigned reading — see the MTU tutorial's Kasiski page: `https://pages.mtu.edu/~shene/NSF-4/Tutorial/VIG/Vig-Kasiski.html`. Describe the method in a paragraph, then give one situation in which Kasiski succeeds where IoC-based key-length recovery is unreliable, and one situation with the reverse. (Your Part 4 data may help.)
4. **IoC as a fitness function: breaking the Enigma plugboard.** The index of coincidence was used in anger against rotor machines. Read the short handout *Hillclimbing the Enigma Machine* (Sullivan & Weierud; in the assignment repository). With the wheel settings correct but *no* plugboard cables connected, only about one letter in twenty-one decrypts correctly, and the output looks like noise.
   (a) Explain why the IoC of the trial decrypt nevertheless rises as correct cables are added — i.e., why it is a usable *fitness score* for a hill climb long before the text is anywhere near readable.
   (b) Explain why IoC stops discriminating once several cables are correct, so that the search must switch to trigram (or bigram) scores — and what would go wrong if you used trigram scores from the very start.
   (c) In what sense is this the same statistic you used in Part 2, and in what sense is it being used differently? (Hint: in Part 2 you *compared* IoC across candidate periods; here you *climb* it.)

   You do not need to implement anything; the ciphertext and handout are there for the curious, and are unsupported and ungraded.

§6 — Review Lab 1: what will be examined

Review Lab 1 is a 15-minute individual session with course staff during the week of September 21. You don't have to bring anything: your submission will be on the screen. This review is graded on a comprehension rubric and produces the multiplier on your assignment credit. You won't be asked about glue code.

Review-critical regions

Three parts of your submission are designated **review-critical**. You will be examined on them as if you had written every line yourself, regardless of how they were produced:

1. **The key-length decision rule** (Part 2): how candidate lengths are scored, and how you choose among them, including ties and multiples.
2. **The column-shift scoring function** (Part 3): the statistic you compare against English letter frequencies, and why it identifies the correct shift.
3. **The behavior of your tool** (Part 4): where it fails and why.

What the session looks like

- **Walkthrough (≈3 min).** Explain your pipeline end to end.
- **Prediction (≈5 min).** Before running anything, predict what your tool will do under a condition you haven't seen, and explain why. Then we run it.
- **Modification (≈5 min).** Make a small change live, or find a flaw that we have planted in a *copy* of your own code.

Some example questions (the real ones will be in this spirit)

Prediction:

- "Here is a 400-letter ciphertext with a 7-letter key. Will `vigenere-keylength` get it? Why or why not?"
- "Same tool, but the plaintext was German. What breaks, and where in your code?"
- "We switch to a 27-symbol alphabet that includes the space character. Which constants in your code are now wrong?"
- "The key is `AAAAA`. What does your Part 2 tool report, and is it wrong?"
- "Walk me through your column-scoring statistic. If it's chi-squared: what are the expected counts, and what happens to the statistic when a column has only 15 letters and the letter Z never appears?"
- "Why does the correct shift *minimize* chi-squared but *maximize* a dot product against English frequencies? Could a wrong shift ever win under one statistic but not the other?"

Modification:

- "Replace your column-scoring statistic with a different one (chi-squared ↔ dot product). Does it still work? Convince me."
- "This copy of your Part 2 reports 14 for a 7-letter key. Find the bug."
- "This copy of your Part 3 gets every key letter right except the first. Find the bug."
- "Sketch how you'd add Kasiski as a second opinion. What changes?"

Rubric and multiplier

Level	You can…	Multiplier
4 — Modify	make correct live changes and find planted flaws	1.0
3 — Predict	correctly predict behavior in new conditions and explain why	0.85
2 — Explain	explain why the review-critical code works (the statistics)	0.6
1 — Describe	say what the code does, but not why it works	0.3
0	cannot describe your own submission, or no-show	0

The Review Lab component of the course grade (25% overall) is scored on the same rubric.

§7 — Required submission notes files

You must include each of these as a separate file. Any missing file means you cannot sign up for a review.

Design note (`design.md`, $\leq$ 1 page): your key-length decision rule (including tie-break and how you handle multiples), your column-scoring statistic, and known failure cases. Not graded for style; used by your interviewer to prepare questions. It's in your interest for it to be accurate.

AI usage note (`ai-usage.md`, a few paragraphs): which tools you used; what you delegated to them; what you verified yourself and how; and one thing you learned from (or caught wrong in) the AI's output. This note is not graded for *how much* you used AI. It will be read by your interviewer, who may start with the parts you say the AI wrote. Omitting or misrepresenting it is an academic integrity violation.

Agent report (*optional, ungraded, anonymous — not part of your submission*): if you used a coding agent with the provided `AGENTS.md`, the agent will offer to write a short report on how well its teaching instructions worked: what it did versus what you did, where the "explain first" rule helped or got in the way, and what it would change about the instructions. We collect these to improve the instructions. **They are anonymous.** The report must not contain your name, JHED, email, file paths, verbatim code, or anything else that identifies you; the agent is instructed to write it that way, and you should check that it complied before sending. Do **not** include it in your Gradescope submission — it goes through a separate anonymous channel announced on Piazza, and the system is built so that we cannot link reports to students (see §9 and the handout `anon-report.pdf` for exactly what that does and doesn't cover).

§8 — What to submit

Your submission is a directory (zip or repo) containing:

```
build.sh              # produces the executables below; must exit 0
bin/vigenere-encrypt
bin/vigenere-decrypt
bin/vigenere-keylength
bin/vigenere-cryptanalyze
bin/vigenere-break    # optional
src/...               # your source
analysis.md           # Part 4
written.md            # Part 5
design.md             # Section 7
ai-usage.md           # Section 7
```

(The optional agent report is **not** submitted here — see §7.)

- `build.sh` runs on the published course container (Ubuntu LTS with current toolchains for Python, Go, Rust, C/C++, Java, and Node; the Dockerfile is in the assignment repository and the exact versions are posted on Piazza). It may compile, or it may simply make wrapper scripts executable. Executables may be scripts with a shebang line.

- If `build.sh` fails or the executables are missing, the autograder score is 0 and you cannot sign up for the Review Lab. **Test your build in the container before the deadline.**
- Executables must read the input file given on the command line, write only the specified output to stdout, and exit 0. Each test has a 60-second timeout.
- Sample plaintext/key/ciphertext triples, the normalization command, and a submission template are provided in the assignment repository.

§9 — Submitting your anonymous agent report

It is perfectly fine to have a coding agent submit your anonymous agent report. In fact, that's exactly how we expect you to use it. The necessary instructions are in `AGENTS.md`, and in the separate handout `anon-report.pdf` (in the assignment repository) if you'd like to do it yourself.

For this assignment the channel is a **pilot**: reports are encrypted to the course public key. They're then signed with your registered key (that you registered in Part 0) and deposited through the anonymous channel using the same `tools/anon-report.py` (its `deposit` mode). The roster will be published on September 14. Deposits travel over Tor, so please **install Tor Browser** (free, `https://www.torproject.org`) before then.

In brief: your report is encrypted on your own machine so that only the instructor can read it, signed with a ring signature that proves it came from an enrolled student without saying which, and deposited over Tor so the server never sees your IP address. What remains is ordinary metadata such as timing and writing style, which we don't inspect. The handout `anon-report.pdf` spells out the steps, the dates (deposit by **Sun Sep 20**; attendance proof by **Fri Sep 25**), and exactly what the guarantee does and doesn't cover. **Read your report before it is sent.**

Authorship and AI usage note *(in the same format this course asks of you).* This handout was written by **Matthew Green** with **Claude** (Anthropic's Fable 5 model, via Claude Code). Delegated to the AI: recovering and comparing the previous versions of this assignment, drafting the text, the LaTeX conversion, and the autograder and review-lab tooling. Done by the human: the assignment's design — what is examined, how, and why — every grading decision, and review and editing of every section. One thing we caught: the first version of the Part 4 experiment grid was too easy — a correct tool never failed anywhere on it — which we discovered only by running a reference implementation through it. The grid you have is the one where the boundary is actually visible.